CS6886 Assignment 5: Optimising Attention on a CPU

All numbers in this report were measured on my laptop (Intel Core Ultra 7 165H, Windows 11, clang 23.1.3 from MSYS2, `-O3 -march=native`). Every time is the median of the 5 timed runs printed by the driver; because the laptop's clock moves around a lot I ran the whole set (all variants, all thread counts) three times and report the median of the three passes. Single-threaded runs were pinned to one P-core (logical CPU 12) with the process affinity mask; without this Windows sometimes moved the thread to an E-core and the same binary was up to 2x slower. FLOP counts use the driver's convention $4N^2d$ (the two matrix products), so exp/max/sum work in the softmax is not counted.

Q1. Machine

The fields are filled in `machine_spec.txt`. Sources: Windows WMI (`Win32_Processor`, `Win32_CacheMemory`, `Win32_PhysicalMemory`), Intel's spec sheet for the 165H, the clang predefined macros (__AVX2__, __FMA__, no __AVX512F__), and three small microbenchmarks I wrote (FMA throughput with 12 independent ymm accumulators, a STREAM-like triad, and a read-bandwidth sweep over working-set sizes). WMI's cache totals match the per-core numbers exactly: L1D 6x48 + 10x32 = 608 KB, L2 6x2 MB + 3x2 MB = 18 MB, L3 24 MB.

Peak FP32 throughput

A P-core (Redwood Cove) has two 256-bit FMA pipes, an E-core (Crestmont) two 128-bit ones:

```
P-core : 2 FMA/cycle x 8 lanes x 2 FLOP = 32 FLOP/cycle
E-core : 2 FMA/cycle x 4 lanes x 2 FLOP = 16 FLOP/cycle

one core (P, 5.0 GHz max turbo)  = 32 x 5.0                          = 160 GFLOP/s
all cores (each at its max turbo) = 6x32x5.0 + 8x16x3.8 + 2x16x2.5
                                  = 960 + 486.4 + 80                 = 1526 GFLOP/s
```

These are nameplate numbers. The laptop cannot run there: the FMA microbenchmark gives ~107-120 GFLOP/s on one pinned P-core (i.e. ~3.4-3.7 GHz effective), 43 GFLOP/s on one E-core (16 FLOP/cycle x ~2.7 GHz, which also confirms the 2x128-bit FMA), 408 GFLOP/s on the 6 P-cores together (68 per core, ~2.1 GHz) and 560-690 GFLOP/s with all 22 threads. So with everything busy the chip is power limited (28 W base power) and clocks drop to about 2 GHz on P-cores. The all-core clocks in machine_spec.txt are derived, not read from a counter: P-cores from the 6-P-core run (408 / (6 x 32) = 2.1 GHz), and E-cores from what is left of the 22-thread total (~630 - 6 x 32 x 2.1 = ~225 GFLOP/s over 10 x 16 FLOP/cycle = ~1.4 GHz). I use 110 GFLOP/s (1 core) and 630 GFLOP/s (all cores) as the 'measured' roofs.

Memory bandwidth and ridge points

```
LPDDR5x-7467, 128-bit bus:  7467e6 transfers/s x 16 B = 119.5 GB/s (theoretical)
measured triad (c = a + 3b, 3 x 256 MB arrays):  1 thread 20.3 GB/s, 22 threads 65.3 GB/s
measured single-core read bandwidth vs size: <=1.5 MB (L1/L2) ~170-185 GB/s, 4 MB ~68 GB/s,
                                             12 MB (L3) ~40 GB/s, >=64 MB (DRAM) ~18.5 GB/s

ridge, one core  = 160  / 119.5 = 1.34 FLOP/B   (measured: 110 / 20.3 = 5.4 FLOP/B)
ridge, all cores = 1526 / 119.5 = 12.8 FLOP/B   (measured: 630 / 65.3 = 9.7 FLOP/B)
```

The theoretical single-core ridge is misleading on this chip: one core can only pull about 20 GB/s from DRAM (limited by how many misses one core can keep in flight), so in practice a single-threaded kernel needs more than ~5 FLOP per DRAM byte to be compute bound.

Q2. Naive baseline

input	time (ms)	GFLOP/s	% of 160 (nameplate)	% of 110 (measured)
N=1024, d=64	110.3	2.43	1.52 %	2.21 %
N=2048, d=64	488.4	2.20	1.37 %	2.00 %
N=4096, d=64	2024.6	2.12	1.33 %	1.93 %
N=4096, d=128	4718.6	1.82	1.14 %	1.65 %

Scaling. Doubling N multiplies the time by 4.4 (1024 to 2048) and 4.1 (2048 to 4096), close to the 4x expected from the N^2d work; going from d = 64 to 128 at N = 4096 multiplies it by 2.33 (2x expected). The time is a bit worse than proportional because the bigger cases fall out of cache (see below), so GFLOP/s drops slightly from 2.43 to

1.82.

Where the time goes. I made an instrumented copy of the naive code with a timer around each step (N = 4096, d = 64, per call): **Q K^T 512 ms, softmax 76 ms, S V 1218 ms**. So about 2/3 of the time is the last product and almost none is the softmax.

1. **Q K^T**: the inner loop is `acc += Q[i][k]*K[j][k]`. Both operands are contiguous, but the loop is a floating-point reduction into one scalar, and without `-ffast-math` the compiler is not allowed to reorder the additions, so it stays scalar: the assembly is one `vfmadd231ss` per iteration into the same register, and every iteration waits for the previous one (4-cycle FMA latency). That is at most 2 FLOP / 4 cycles = 0.5 FLOP/cycle, i.e. 2.0-2.5 GFLOP/s at 4-5 GHz (scalar code draws much less power than my AVX FMA test, so the core turbos higher), which is exactly the range naive runs at. It is *latency bound*, using 1 of 8 SIMD lanes and 1 of 2 FMA pipes with one op in flight instead of 8.

2. **S V**: same scalar dependency chain, and in addition the inner loop walks `V[j][k]` down a column, stride d x 4 = 256 B, so every single multiply touches a new cache line. V is 1 MB (fits in the 2 MB L2) so these are L1 misses served from L2, but the extra latency makes this step 2.4x slower than Q K^T.

3. **Softmax**: one row (16 KB) stays in L1 for its three passes, so only the scalar `expf` calls matter (16.7 M calls, ~4.5 ns each).

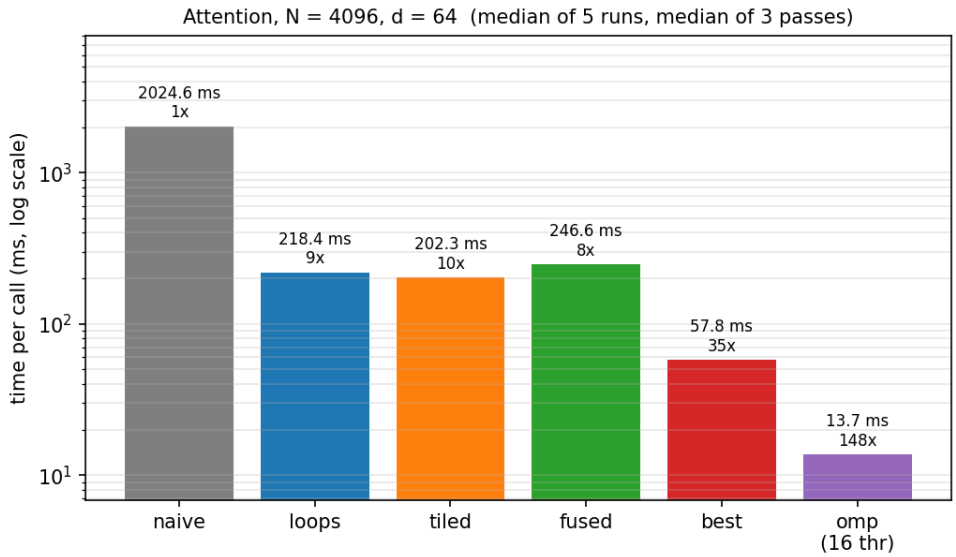
The N x N matrix S (64 MB at N = 4096) is written once, read and written by the softmax and read again, i.e. ~256 MB of DRAM traffic. At 20 GB/s that is only ~13 ms out of 2 s, so naive is not memory bound at all; it is limited by the scalar dependency chains.

Q3. Optimisations

All variants pass `make check` on all four inputs (errors 5e-7 to 6e-6, limit 1e-4). I also generated an extra N = 1000, d = 40 input (not multiples of 16) to test the edge/padding code of tiled, best and omp; all agreed with naive to ~1e-6.

variant	N=1024 d=64	N=2048 d=64	N=4096 d=64	N=4096 d=128
naive	110.3 ms 2.4 GF/s, 1.5% 1.0x	488.4 ms 2.2 GF/s, 1.4% 1.0x	2024.6 ms 2.1 GF/s, 1.3% 1.0x	4718.6 ms 1.8 GF/s, 1.1% 1.0x
loops	16.4 ms 16.4 GF/s, 10.3% 6.7x	59.5 ms 18.0 GF/s, 11.3% 8.2x	218.4 ms 19.7 GF/s, 12.3% 9.3x	392.4 ms 21.9 GF/s, 13.7% 12.0x
tiled	8.4 ms 31.8 GF/s, 19.9% 13.1x	47.7 ms 22.5 GF/s, 14.1% 10.2x	202.3 ms 21.2 GF/s, 13.3% 10.0x	283.6 ms 30.3 GF/s, 18.9% 16.6x
fused	14.2 ms 19.0 GF/s, 11.8% 7.8x	54.1 ms 19.8 GF/s, 12.4% 9.0x	246.6 ms 17.4 GF/s, 10.9% 8.2x	496.9 ms 17.3 GF/s, 10.8% 9.5x
best	3.4 ms 78.3 GF/s, 48.9% 32.2x	13.8 ms 77.9 GF/s, 48.7% 35.4x	57.8 ms 74.3 GF/s, 46.5% 35.0x	107.8 ms 79.7 GF/s, 49.8% 43.8x

Single thread, pinned to a P-core. Each cell: time, GFLOP/s and % of the 160 GFLOP/s nameplate peak, speedup over naive.



plots/time_N4096_d64.png (the other three inputs are in plots/).

(a) attn_loops.c: loop reordering

I transpose K once into Kt (d x N, O(Nd) cost) and change step 1 to i-k-j: `S[i][j] += q[i][k]*Kt[k][j]`. Now the inner loop has no reduction: each j is independent and both arrays are unit-stride, so clang vectorises it (8 wide, unrolled 4x; checked with `-Rpass=loop-vectorize`). Step 3 becomes i-j-k: `O[i][k] += S[i][j]*V[j][k]`, inner loop over a row of V. The softmax division is replaced by a multiply with 1/sum (vectorises), but `expf` remains scalar because there is no vector math library.

Result: 9.3x faster at N = 4096, d = 64. Phase times from an instrumented run: Q K^T ~72 ms, softmax ~89 ms, S V ~71 ms. Each of the matrix products now runs at ~30 GFLOP/s, but the inner loops do one load of Kt (or V) and a load+store of the S (or O) row per FMA: they are limited by L1/L2 load/store bandwidth, not by the FMA units. The softmax (scalar expf) is now the biggest single step.

(b) attn_tiled.c: cache and register tiling

Both products are computed in 4 x 16 output tiles kept in a local array `acc[4][16]`; clang turns it into 8 ymm accumulators (I checked the assembly: 8 `vfmadd231ps` per k step, no loads/stores of acc inside the loop). Every loaded vector of Kt is used for 4 rows and every broadcast of Q for 16 columns, so the load:FMA ratio drops from ~2:1 to ~0.75:1. The j loop is blocked (JB = 128) so the Kt panel (d x 128 x 4 B = 32 KB at d = 64) stays in L1 while all rows pass over it; for S V the V panel is blocked the same way and O tiles are reloaded per j block.

Result: S V went from 71 to ~32 ms and Q K^T from 72 to ~59 ms, but total speedup over loops is small at d = 64 (202.28 ms vs 218.40 ms) and larger at d = 128 (284 vs 392 ms). The reasons: (1) the scalar-expf softmax (~75 ms) is unchanged and is now ~40% of the time; (2) S still goes to DRAM and back: Q K^T writes 64 MB, and the 4 x 16 tile writes S with stride N, so the store stream and the RFO reads hit DRAM; (3) 8 accumulators are just enough to hide the 4-cycle FMA latency on 2 ports, with no slack. At d = 128 the matrix products are a larger share of the work so tiling helps more.

(c) attn_fused.c: row-wise fusion

For each query row: `s = q Kt` (N floats), max, exp and sum on s, then `o = s V` and o *= 1/sum. The N x N matrix is never stored; the score row (16 KB) stays in L1 and DRAM traffic drops from ~260 MB to ~6 MB.

I expected this to be the fastest of the three, but it is **slower than tiled** at N = 4096 (247 ms vs 202 ms). The reason is that removing DRAM traffic did not matter (S was never the bottleneck, see Q2) while fusion forces a matrix-vector structure: for every row, all of Kt and V (2 x N x d x 4 B = 2 MB at d = 64, 4 MB at d = 128) are streamed again to do only 4Nd FLOPs, i.e. **0.5 FLOP per byte** from cache. 2 MB plus everything else does not fit in the 2 MB L2, so it comes from L3 at ~40-60 GB/s: 4096 rows x 2 MB = 8 GB / ~50 GB/s = ~160 ms, plus ~75 ms of scalar expf, which matches the measured 247 ms. At N = 1024 (Kt + V = 0.5 MB, fits in L2) fused is fine. So fusion alone just moves the bottleneck from DRAM (which was not the problem) to the L2/L3 bandwidth (which is).

(d) attn_best.c: what I did and why it works

The lesson from (b) and (c) is: fuse to keep S out of DRAM, but process *several* rows together so each Kt / V element pulled from cache is reused many times. attn_best.c does:

- **Block fusion.** BR = 48 query rows at a time. Their score block (48 x N x 4 B = 768 KB at N = 4096) lives in the private 2 MB L2. Each Kt / V element is reused 48 times, so cache-level intensity goes from 0.5 (fused) to 24 FLOP/B, above the single-core L3 ridge (110 / ~45 = 2.4).
- **Hand-written AVX2/FMA micro-kernel** (6 rows x 16 columns = 12 ymm accumulators, 2 loads + 6 broadcasts per 12 FMAs), used for both Q K^T and P V. 12 independent accumulators are more than the 8 needed to cover latency x throughput (4 x 2), and 12 + 2 + 1 = 15 of the 16 ymm registers are used. j is blocked by 256 so the Kt / V panel stays in L1/L2 while the 48 rows go over it.
- **Vectorised exp** (Cephes polynomial: range reduction with 2^n by exponent bit manipulation, degree-5 polynomial, ~1-2 ulp). This replaced the scalar libm expf that cost ~75 ms in tiled. Max and sum are vectorised too.
- **Small things**: 1/sqrt(d) is folded into the copy of the Q block, 1/sum is applied to the d outputs instead of the N probabilities, and inputs are zero-padded to multiples of 16 (padded score columns set to -inf) so the kernel has no tail code.

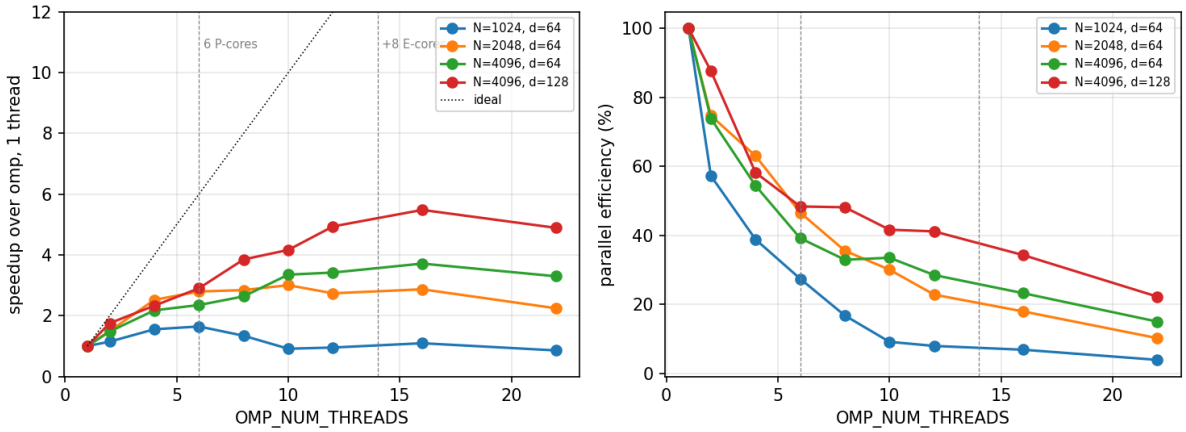
Result: 74.3-79.7 GFLOP/s on the four inputs, **32x-44x faster than naive** and 3.5x faster than tiled at N = 4096, d = 64. That is 46% of the 160 GFLOP/s nameplate peak and ~68% of what my FMA loop reaches on one core (110). What is left: the exp/max/sum work that is not counted as FLOPs (~10-15% of the time by my estimate: 16.7 M exps x ~20 vector ops / 8 lanes), broadcasts that use load ports, the transpose/pad copies, and the frequency dropping when the core runs hot. I also tried BR in {24, 48, 96} and JB in {64, 128, 256}; differences were within the +-10% run-to-run noise, BR = 48 / JB = 256 was marginally best.

Q4. OpenMP

attn_omp.c uses the same kernels as attn_best.c. The loop over row blocks is a `#pragma omp for schedule(dynamic, 1)`; blocks are fully independent (own scores, softmax and output rows) so there is no reduction and no locking. Each thread allocates its own qb / S / ob scratch once, so its score block stays in its own L2. The K transpose and V padding are also split across threads. I reduced BR to 24 so N = 1024 still has 43 blocks for up to 22 threads; dynamic scheduling matters here because P-cores and E-cores run at very different speeds. Threads were not pinned (Windows places them), OMP_NUM_THREADS was varied.

threads	N=1024 d=64	N=2048 d=64	N=4096 d=64	N=4096 d=128
1	3.54 ms 76 GF/s 1.00x, eff 100%	15.26 ms 70 GF/s 1.00x, eff 100%	50.92 ms 84 GF/s 1.00x, eff 100%	104.95 ms 82 GF/s 1.00x, eff 100%
2	3.09 ms 87 GF/s 1.15x, eff 57%	10.21 ms 105 GF/s 1.49x, eff 75%	34.47 ms 125 GF/s 1.48x, eff 74%	59.86 ms 144 GF/s 1.75x, eff 88%
4	2.28 ms 118 GF/s 1.55x, eff 39%	6.05 ms 177 GF/s 2.52x, eff 63%	23.40 ms 184 GF/s 2.18x, eff 54%	45.09 ms 191 GF/s 2.33x, eff 58%
6	2.15 ms 125 GF/s 1.64x, eff 27%	5.46 ms 197 GF/s 2.80x, eff 47%	21.66 ms 198 GF/s 2.35x, eff 39%	36.16 ms 238 GF/s 2.90x, eff 48%
8	2.64 ms 102 GF/s 1.34x, eff 17%	5.37 ms 200 GF/s 2.84x, eff 36%	19.33 ms 222 GF/s 2.63x, eff 33%	27.25 ms 315 GF/s 3.85x, eff 48%
10	3.88 ms 69 GF/s 0.91x, eff 9%	5.08 ms 211 GF/s 3.00x, eff 30%	15.20 ms 283 GF/s 3.35x, eff 34%	25.21 ms 341 GF/s 4.16x, eff 42%
12	3.72 ms 72 GF/s 0.95x, eff 8%	5.58 ms 193 GF/s 2.74x, eff 23%	14.89 ms 288 GF/s 3.42x, eff 28%	21.26 ms 404 GF/s 4.94x, eff 41%
16	3.24 ms 83 GF/s 1.09x, eff 7%	5.32 ms 202 GF/s 2.87x, eff 18%	13.71 ms 313 GF/s 3.71x, eff 23%	19.15 ms 449 GF/s 5.48x, eff 34%
22	4.14 ms 65 GF/s 0.86x, eff 4%	6.80 ms 158 GF/s 2.24x, eff 10%	15.45 ms 278 GF/s 3.30x, eff 15%	21.46 ms 400 GF/s 4.89x, eff 22%

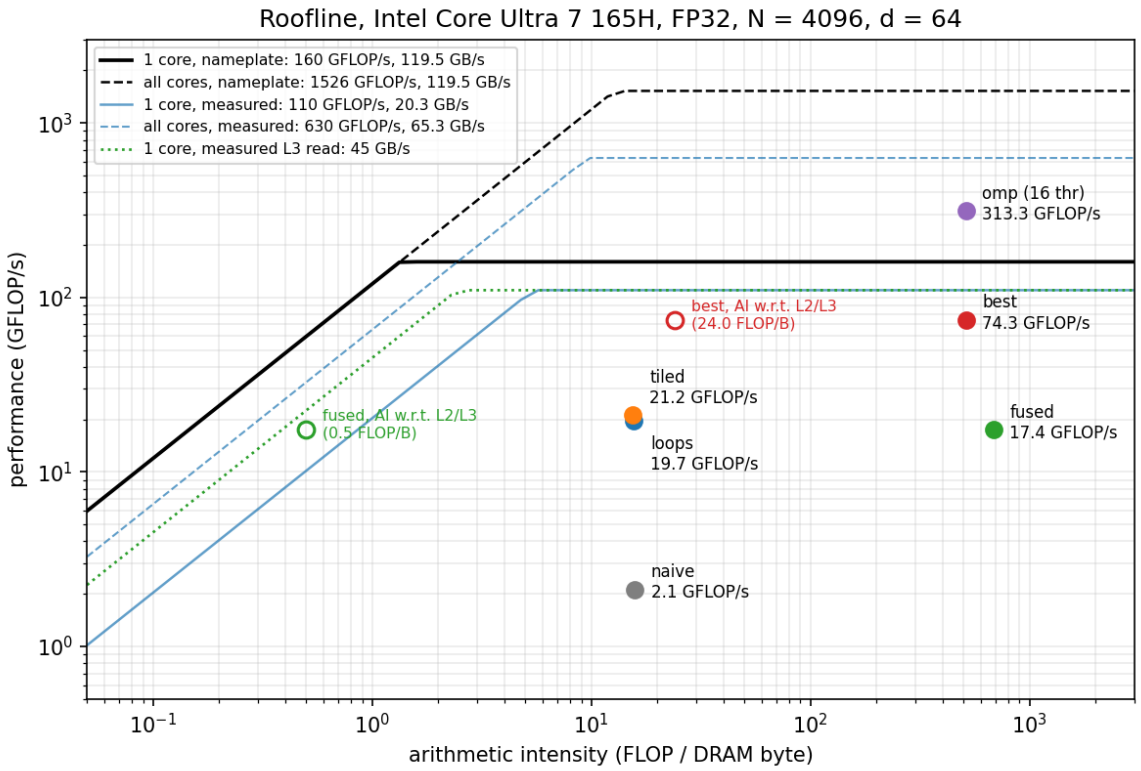
Speedup and efficiency are relative to attn_omp with 1 thread.



Best results: N = 4096, d = 128: 19.1 ms with 16 threads (449 GFLOP/s, 5.5x over 1 thread, 246x over naive); N = 4096, d = 64: 13.7 ms with 16 threads (3.7x, 148x over naive). The scaling is clearly sub-linear and flattens after ~8-12 threads. My explanation, roughly in order of importance:

- **Power limit / clock drop.** The FMA microbenchmark shows the same shape: 110-120 GFLOP/s for 1 thread but only 68 GFLOP/s per P-core with 6 P-cores busy, and 560-690 GFLOP/s total with everything busy (vs 1526 nameplate). A kernel that is ~70% of peak on one core cannot scale past what the chip can sustain at 28 W.
- **Heterogeneous cores.** Only 6 cores are P-cores. Beyond 6 threads, the extra threads end up either on the Hyper-Thread siblings of busy P-cores (which share the two FMA pipes, so for an FMA-bound kernel they add little) or on E-cores (~40% of a P-core's FMA throughput) and LP-E cores (slower still, and outside the L3). With dynamic scheduling the slow threads just take fewer blocks, so 22 threads are worth maybe 8-9 'P-core equivalents', which is close to the best speedups I see; efficiency per thread therefore looks bad even though the cores are busy.
- **Shared L3 bandwidth.** Every block re-reads Kt and V (2 MB at d = 64) from L3; with BR = 24 that is ~350 MB per call at N = 4096, i.e. ~25 GB/s at 14 ms, a fair fraction of the ~65-100 GB/s the L3/ring can deliver to all cores. d = 128 has twice the FLOPs per block for the same structure and scales better (5.5x vs 3.7x).
- **Small problems.** At N = 1024 one call is only ~3.5 ms of work. Waking the threads, the barrier after the transpose and the last slow blocks on E-cores are a big fraction, so the best is 6 threads (1.6x) and more threads get slower. Windows' scheduler moving threads between core types also shows up as large pass-to-pass variation (e.g. 8 threads at N = 1024 ranged 1.9-3.0 ms).

Q5. Roofline



plots/roofline.png. Solid markers: intensity w.r.t. DRAM bytes. Hollow markers: fused and best placed at their intensity w.r.t. L2/L3 bytes, against the measured single-core L3 roof (dotted).

How the intensities were computed (N = 4096, d = 64, FLOPs = $4N^2d$ = 4.29 GFLOP). I count the compulsory DRAM bytes implied by each variant's data structures, assuming that anything that fits in L2 (Q/K/V rows, one score row, the Kt/V panels) is not re-read from DRAM:

```
Q, K, V in + O out              : 4 N d x 4 B            =   4.2 MB  (all variants)
naive : + S written, read+written by softmax, read by S V
        4 N^2 x 4 B = 268 MB    -> 4.29e9 / 272e6       =  15.8 FLOP/B
loops : naive + Kt copy (2 N d x 4 B)                   =  15.6 FLOP/B
tiled : loops + scaled Q copy                           =  15.5 FLOP/B
fused : no S;  4.2 MB + Kt copy 2.1 MB                  = 683   FLOP/B
best  : no S;  4.2 MB + Kt and padded V copies 4.2 MB   = 512   FLOP/B   (omp: same)
cache-level (bytes re-read from L2/L3 per row / block of rows):
fused : 4 N d FLOP per (Kt + V) = 2 N d x 4 B           =   0.5 FLOP/B
best  : 48 rows share one pass over Kt + V              =  24   FLOP/B
```

Exact values for every input are in the arithmetic_intensity column of results.csv. They are 'model' numbers, not measured with hardware counters (I did not have VTune/perf on this machine).

What limits each variant. All variants are to the right of the single-core ridge (1.34 nameplate, 5.4 measured), so by DRAM traffic alone every one of them should be compute bound. The gaps below the roof are therefore explained by in-core and cache effects:

- **naive** (2.1 GFLOP/s, ~1.3% of peak): far below the roof because of scalar code and the loop-carried add dependency (latency bound), plus the strided V access. Memory is not the limit.
- **loops / tiled** (~20 GFLOP/s): vectorised but limited by L1/L2 load-store traffic in the inner loops (loops) and by the scalar expf softmax plus S round trips (tiled). Their DRAM intensity of ~16 FLOP/B is above the ridge, so DRAM is not what stops them either; at higher thread counts S traffic (64 MB per pass) would start to matter.
- **fused** (17 GFLOP/s): huge DRAM intensity but only 0.5 FLOP/B w.r.t. L2/L3. Placed at that intensity it sits right under the measured L3 roof (0.5 x 45 = 22 GFLOP/s): it is cache-bandwidth bound.
- **best** (74 GFLOP/s, 46% of nameplate, ~68% of measured 1-core peak): 24 FLOP/B from cache, above the L3 ridge, so it is compute bound; what is left is the uncounted softmax work and frequency.
- **omp** (313 GFLOP/s at 16 threads): compute bound too, but compared with the all-core roofs it reaches about half of the measured 630 GFLOP/s and ~20% of the nameplate 1526, because of the power limit and the P/E-core mix described in Q4.

Notes on how I ran things

Toolchain: MSYS2 UCRT64 with clang 23.1.3, llvm-openmp 23.1.3 and GNU make 4.4.1; the Makefile's CC points to `/c/msys64/ucrt64/bin/clang` if it exists and to plain `clang` otherwise; everything else in the Makefile is unchanged. attn_best.c and attn_omp.c use AVX2 intrinsics only when __AVX2__ and __FMA__ are defined; otherwise (e.g. on an ARM machine) a plain C version of the same kernels is compiled, which gives the same results but is slower. I checked that it passes against naive too (built without -march=native). All timings in this report are from the AVX2 path. The driver code in every variant is unchanged (on Windows it prints the full path of the exe, because it looks for '/' in argv[0]; harmless).